



# PROPOSTA EXECUTIVA - SOLUÇÃO BIG DATA

Modelagem de *Data Warehouse* (Parte III)

Integrantes: Ana Clara de Jesus Nascimento, Mariana Rocha de Moraes,  
Matheus Mangia Terra Suzano, Paulo Henrique Cotta de Assumpção,  
Pedro Nunes de Oliveira Pessanha, Ryan Domingos dos Santos Saraiva  
DRE: 124388471, 122147926, 123378716, 122156268, 123177330, 123137429

RIO DE JANEIRO  
2026

# Sumário

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução e Objetivos</b>                                           | <b>2</b>  |
| <b>2</b> | <b>Computação de Borda e Pré-processamento</b>                          | <b>3</b>  |
| 2.1      | Stack Tecnológico: O que escolhemos e por que não outros? . . . . .     | 3         |
| <b>3</b> | <b>Camada de Ingestão e Barramento de Eventos</b>                       | <b>4</b>  |
| 3.1      | Stack Tecnológico: O que escolhemos e por que não outros? . . . . .     | 4         |
| <b>4</b> | <b>Camada de Processamento de Dados</b>                                 | <b>4</b>  |
| 4.1      | Divisão dos Motores de Processamento (Spark, Flink e MapReduce) . . . . | 5         |
| 4.2      | Arquitetura de Hardware (Nuvem - Processamento) . . . . .               | 5         |
| <b>5</b> | <b>Camada de Armazenamento e Persistência</b>                           | <b>6</b>  |
| <b>6</b> | <b>O Dashboard e a IA</b>                                               | <b>6</b>  |
| 6.1      | Detalhamento dos Painéis do Dashboard . . . . .                         | 6         |
| 6.2      | Especificação Funcional do Protótipo do Chatbot de Voz (IA Concierge) . | 7         |
| <b>7</b> | <b>Governança, Segurança e Conformidade (LGPD)</b>                      | <b>9</b>  |
| <b>8</b> | <b>Estimativa de Custos Financeiros da Nuvem</b>                        | <b>9</b>  |
| <b>9</b> | <b>Conclusão</b>                                                        | <b>10</b> |

# 1 Introdução e Objetivos

Este documento apresenta a Proposta Executiva e o desenho da arquitetura de Big Data, Internet das Coisas (IoT) e Inteligência Artificial projetada para o consórcio de seis locadoras de veículos. Como evolução natural das infraestruturas transacional (OLTP) e analítica (Data Warehouse) consolidadas nas Partes I e II, o ecossistema de dados da locadora agora expande suas fronteiras para incorporar a recente aquisição de frotas de "carros inteligentes". Estes novos veículos, equipados com sensores telemáticos, transmitem sua localização e status mecânico a cada três segundos, gerando um volume massivo de dados em fluxo contínuo (*streaming*).

O objetivo que norteia esta nova solução é transcender a análise histórica de dados e capacitar o consórcio a operar de forma ágil, preditiva e em tempo real. A arquitetura descrita a seguir foi desenhada para atender a três grandes necessidades de negócio:

1. **Monitoramento Operacional Instantâneo:** Suportar a ingestão de dados em alta velocidade para alimentar dashboards em tempo real, permitindo a identificação imediata de acidentes ou falhas mecânicas severas.
2. **Precificação Dinâmica e Gamificação:** Avaliar o comportamento de direção do cliente em tempo real (como excesso de velocidade ou condução econômica) para gerar multas ativas ou descontos baseados em desgaste de frota.
3. **Assistência Interativa (Concierge de Viagem):** Integrar Inteligência Artificial Generativa orientada a dados do próprio domínio (utilizando a técnica de *Retrieval-Augmented Generation* – RAG) para interagir via voz com o condutor, recomendando rotas e pontos de interesse de forma segura e sem alucinações.

A necessidade de uma arquitetura baseada neste ecossistema justifica-se pela aderência do projeto aos **5 V's do Big Data**. A frota de veículos conectados gera um **Volume** massivo de dados de telemetria diariamente. A necessidade de alertas de acidentes e roteamento exige processamento em tempo real, caracterizando a **Velocidade**. Lidar com dados transacionais estruturados, arquivos JSON dos sensores IoT e áudio desestruturado do cliente evidencia a **Variedade**. A utilização de *Edge Computing* para filtrar falsos positivos diretamente no veículo garante a **Veracidade** da informação. Por fim, a geração de **Valor** consolida-se com a criação de novas receitas através do assistente de IA e a redução de custos operacionais.

Cabe ressaltar que a modelagem relacional desenvolvida nas etapas anteriores deste trabalho (utilizando MySQL para o *Data Warehouse* analítico) atua como o **MVP (Minimum Viable Product)** e projeto-piloto desta iniciativa. A proposta executiva a seguir apresenta a visão de escala necessária para suportar o crescimento exponencial da volumetria de dados de telemetria quando o projeto for expandido para toda a rede de empresas associadas, sem descontinuar a infraestrutura relacional (MySQL) já validada nas etapas anteriores.

Toda a modelagem, divisão de camadas e escolhas de *stack* tecnológico (como Apache Kafka, Spark, MongoDB e Redis) justificadas nas seções seguintes foram orientadas pela necessidade de aliar processamento de baixíssima latência (*Hot Path*) com a retenção histórica escalável (*Cold Path*), conectando o novo mundo do Big Data ao legado do nosso Data Warehouse.

## 2 Computação de Borda e Pré-processamento

Para evitar o estrangulamento da rede e custos exorbitantes de tráfego na *cloud*, a primeira camada da nossa arquitetura ocorre dentro do próprio veículo.

- **Arquitetura de Hardware (Borda):** Cada veículo da frota será equipado com um Gateway de Borda industrial baseado na arquitetura **ARM Cortex-A72 (Quad-core, 1.5GHz, com 4GB de RAM LPDDR4)**, integrado com módulos de barramento CAN (para ler dados do motor) e modems automotivos **4G/5G LTE Categoria 4** com antenas de alto ganho. Este *hardware* robusto garante capacidade computacional para executar rotinas locais de filtragem sob temperaturas extremas (-40°C a 85°C) e vibração constante.
- **Arquitetura de Software (Borda):** O computador de bordo funciona como a primeira barreira analítica, dividindo o processamento local em dois fluxos estritos:
  - **Fluxo de Rotina (Batch/Stream Agregado):** O Gateway de Borda acumula leituras dos sensores numa janela de tempo local (*Tumbling Window* de 1 segundo). O sistema calcula médias, máximas e mínimas de RPM, velocidade e consumo, compactando estes dados e enviando-os em lotes a cada 10 segundos para a nuvem.
  - **Fluxo de Eventos Críticos (Alerta Imediato):** O sistema possui gatilhos (*thresholds*) baseados em anomalias (ex: desaceleração brusca superior a 2.5G detectada pelo sensor inercial/acelerômetro). Ao ser acionado, este fluxo interrompe a transmissão de rotina, alocando a banda total para enviar os metadados do acidente e telemetria exata em milissegundos.

### 2.1 Stack Tecnológico: O que escolhemos e por que não outros?

- **Protocolo de Comunicação: MQTT (Message Queuing Telemetry Transport)**
  - *Por quê?* É o padrão para IoT. É leve, consome pouca energia e foi desenhado para redes móveis instáveis. Ele possui o recurso de *Last Will and Testament* e níveis de QoS (Quality of Service) para garantir a entrega da mensagem assim que a conexão for restabelecida após túneis ou áreas de sombra.
  - *Por que não HTTP/REST?* O HTTP possui um cabeçalho muito pesado (*overhead* elevado). Se o carro enviasse um pedido POST HTTP a cada 3 segundos, gastaríamos mais largura de banda transmitindo metadados de rede do que dados reais dos sensores.
- **Formato de Serialização: Apache Avro**
- **Por quê?** O Avro compacta os dados em formato binário compacto, reduzindo o consumo de dados móveis. Ele carrega a estrutura (*schema*) de forma separada, facilitando a validação e evolução do esquema no Data Center sem quebrar os sistemas.
  - *Por que não XML ou JSON?* XML é excessivamente verboso. O JSON repete o nome das colunas/chaves em cada linha enviada, multiplicando o custo de armazenamento e tráfego de rede por milhares de veículos.

### 3 Camada de Ingestão e Barramento de Eventos

Para a camada de ingestão, a arquitetura utilizará um sistema de *Pub/Sub* (*Publish/Subscribe*) focado em *streaming* de eventos. O sistema atuará como um grande barramento de dados (*Message Broker*), desacoplando os veículos (produtores) dos sistemas analíticos (consumidores).

- **Organização em Tópicos:** As informações recebidas via gateway MQTT na nuvem serão roteadas para Tópicos distintos no barramento:
  - `topic_telemetria_rotina`: Fila particionada para receber os pacotes compactados de rotina. Focada em alta taxa de transferência (*High Throughput*).
  - `topic_alertas_criticos`: Fila isolada e prioritária para receber os dados de acidentes. Focada em baixíssima latência (*Low Latency*).
- **Resiliência e Particionamento:** Cada tópico será particionado utilizando o `ID_Veiculo` como chave de partição, garantindo que as mensagens de um mesmo carro caiam sempre no mesmo nó e mantenham a ordenação cronológica estrita via *timestamp*. Cada partição terá um fator de replicação igual a 3 espalhado pela nuvem para tolerância a falhas de servidores.

#### 3.1 Stack Tecnológico: O que escolhemos e por que não outros?

##### Tecnologia Principal: Apache Kafka

- *Por quê?* O Kafka grava as mensagens em disco de forma sequencial, suportando a ingestão de milhões de eventos por segundo sem estrangulamentos. Como armazena o histórico em um log imutável distribuído, permite que diferentes sistemas consumam os mesmos dados em tempos diferentes (em tempo real ou em lote).
- *Por que não RabbitMQ ou ActiveMQ?* O RabbitMQ elimina a mensagem imediatamente após o consumo e mantém as filas na memória RAM, o que causaria perda de dados ou estouro de memória sob a volumetria massiva de telemetria contínua gerada por uma frota de milhares de veículos.

### 4 Camada de Processamento de Dados

Para atender às demandas analíticas do consórcio, realizou-se uma avaliação crítica entre as duas principais filosofias de processamento de Big Data:

| Critério de Comparação   | Arquitetura Kappa                                                       | Arquitetura Lambda                                                              |
|--------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Abordagem Central        | Tudo é tratado como um fluxo contínuo (Stream).                         | Divisão em duas vias independentes: Batch (Lote) e Speed (Tempo real).          |
| Processamento Histórico  | Exige reprocessar todo o log de eventos do início caso as regras mudem. | Processamento recorrente e estável sobre dados consolidados em repouso.         |
| Complexidade de Código   | Código único para processamento, facilitando manutenção.                | Exige manter duas bases de código (uma para lote e outra para tempo real).      |
| Garantia de Consistência | Depende de tolerância a falhas complexa no motor de stream.             | Excelente. A camada Batch corrige eventuais atrasos ou falhas da via rápida.    |
| Aderência ao Negócio     | Ideal para sistemas puramente operacionais/reativos.                    | Perfeita para a Locadora: Exige auditoria financeira precisa e alertas ao vivo. |

Tabela 1: Comparação entre as arquiteturas Kappa e Lambda

- **Decisão pela Arquitetura Lambda:** Optou-se pela **Arquitetura Lambda**. A via rápida (*Hot Path*) atende ao monitoramento instantâneo e à segurança do condutor, enquanto a via tradicional (*Cold Path*) garante auditoria sem falhas para

o faturamento de fechamento de contratos, relatórios de diretoria e sincronização com o legado.

- **Ponte com o Data Warehouse Legado:** Os dados processados no *Cold Path* alimentam tabelas agregadas armazenadas em formato *Delta Lake* (que garante transações ACID sobre arquivos Parquet). Madrugada adentro, esses dados refinados (como quilometragem acumulada e dias rodados) alimentam periodicamente a tabela fato fato\_locacao (ou uma nova tabela fato agregada de telemetria), preservando o modelo dimensional estabelecido na Parte II; já os atributos puramente descritivos do veículo, como categoria ou status atual da frota, continuam residindo na dim\_veiculo.

## 4.1 Divisão dos Motores de Processamento (Spark, Flink e MapReduce)

A camada de processamento utiliza três tecnologias distintas para propósitos específicos e complementares:

- **Apache Flink (*Hot Path* - True Streaming):** Atua diretamente na escuta do topic\_alertas\_criticos do Kafka. Processa cada evento individualmente em milissegundos (*event-driven*), operando em janelas deslizantes (*Sliding Windows*). É o responsável por disparar alertas imediatos para a central de emergência e calcular o score instantâneo de direção (freadas bruscas, acelerações severas).
- **Apache Spark (Micro-batching / Machine Learning):** Atua no processamento em micro-lotes de alta eficiência e na orquestração do *Cold Path*. É utilizado na **Manutenção Preditiva**: o Spark consome o histórico de sensores armazenado no Data Lake e aplica modelos de regressão e classificação (via Spark MLlib) para prever a probabilidade de falha mecânica de componentes (como temperatura do bloco e desgaste de freios) antes que ocorram, agendando paradas preventivas na oficina de forma automatizada.
- **Hadoop MapReduce (Processamento Histórico de Exaustão):** Embora o Spark seja significativamente mais rápido para algoritmos iterativos em memória, o MapReduce é mantido e justificado na arquitetura para **processamento em lote em nível de petabytes de dados brutos históricos (Exaustão)**. O MapReduce realiza tarefas pesadas de agregação em disco, transformações estruturais em massa e limpeza de logs brutos antigos (com mais de 1 ano de armazenamento) no HDFS/S3 durante períodos de inatividade. O MapReduce é imbatível na resiliência de jobs em lote de longa duração que excedem a capacidade de memória dos clusters Spark, garantindo que o processamento em disco nunca caia por falta de RAM (*Out of Memory*).

## 4.2 Arquitetura de Hardware (Nuvem - Processamento)

O processamento na nuvem será executado em um cluster gerenciado composto por:

- **1 Nó Mestre (Master Node):** Instância computacional otimizada para computação (**8 vCPUs, 32GB RAM**), responsável pela coordenação dos *jobs* e alocação de recursos.

- **6 Nós de Trabalho (Worker Nodes):** Instâncias otimizadas para memória e processamento paralelo (**16 vCPUs, 64GB RAM e 500GB de SSD NVMe local para cache de disco** por nó), totalizando 96 núcleos de processamento e 384GB de memória RAM distribuída para suportar a computação em memória do Spark e Flink.

## 5 Camada de Armazenamento e Persistência

A arquitetura adota o conceito de persistência poliglota, utilizando diferentes tecnologias NoSQL e de armazenamento distribuído para otimizar o custo e a velocidade de leitura/escrita:

- **Amazon S3 (ou HDFS) – Data Lake:** Armazena o volume bruto total de telemetria e arquivos históricos em formato colunar altamente compactado (Parquet/Delta), servindo de base para o *Cold Path* e jobs do MapReduce/Spark.
- **Apache Cassandra (NoSQL Wide-Column Store):** Responsável por aguentar a altíssima taxa de escrita contínua da telemetria de rotina da frota. Sua arquitetura descentralizada e escalável horizontalmente (*masterless*) permite que novos nós sejam adicionados sem interrupção do sistema.
- **Redis (NoSQL Key-Value em Memória):** Funciona como a base de dados de tempo real para o *Hot Path*. Armazena apenas o último estado conhecido de cada veículo (ID, Placa, Latitude, Longitude, Velocidade Atual e Status). Como opera puramente em memória RAM, entrega leituras em microsegundos para o mapa ao vivo.
- **MongoDB Atlas (NoSQL Document Store):** Armazena documentos JSON semiestruturados contendo o perfil de condução dos motoristas, prontuários de manutenção da frota, catálogos georreferenciados de pontos turísticos/restaurantes e os índices vetoriais necessários para o funcionamento da Inteligência Artificial.

## 6 O Dashboard e a IA

### 6.1 Detalhamento dos Painéis do Dashboard

O monitoramento e a gestão do consórcio serão consolidados em uma interface web rica de gerenciamento, alimentada por conexões bi-direcionais contínuas via **WebSockets** conectadas diretamente ao Redis e ao Flink. O Dashboard é composto por seis painéis funcionais específicos:

1. **Painel de Viagens em Andamento:** Exibe um mapa vetorial interativo renderizando a posição geoespacial em tempo real de todos os veículos ativos, com filtros por modelo, velocidade atual e região metropolitana. O painel integra ainda camadas de dados externos via APIs de terceiros (ex: Google Maps Traffic API e provedores meteorológicos como OpenWeather), sobrepondo no mapa informações de engarrafamentos, condições climáticas adversas e eventos ou alertas urbanos (bloqueios de via, manifestações, obras) que possam impactar a circulação da frota e a definição de rotas.

2. **Painel de Reservas:** Mostra gráficos de linha temporais com a taxa de ocupação preditiva da frota, cruzando reservas abertas, veículos disponíveis nos pátios e estimativas de devolução nas próximas 24 horas.
3. **Painel de Locações:** Painel analítico gerencial com indicadores de faturamento bruto acumulado no dia, ticket médio por categoria e ranking das agências com maior giro de contratos.
4. **Painel de Cobranças e Financeiro:** Monitora em tempo real a aplicação do faturamento dinâmico. Exibe alertas automáticos quando um cliente atinge pontuações de risco na condução, aplicando taxas por desgaste acelerado da frota diretamente na pré-fatura do contrato ativo.
5. **Painel de Emergências:** Painel de prioridade máxima com alertas visuais e sonoros piscando na tela quando o tópico `topic_alertas_criticos` do Kafka reporta uma desaceleração severa ou ativação de *airbag*. Apresenta instantaneamente a localização do carro, telefone do condutor e o botão de acionamento de socorro médico. Em caso de sinistro confirmado, o painel dispara automaticamente a geração de um dossiê digital de regulação, compilando telemetria dos 60 segundos anteriores e posteriores ao evento (velocidade, ângulo de impacto, frenagem), imagens das câmeras 360° embarcadas, timestamp, geolocalização exata e dados do condutor, empacotados em um relatório padronizado para acelerar o processo junto à seguradora.
6. **Painel de Oficina e Manutenção:** Lista os veículos cujo processamento preditivo do Apache Spark identificou comportamento mecânico anômalos. Exibe o nível de severidade e o componente com falha iminente, gerando ordens de serviço automáticas e bloqueando o carro para novas reservas no sistema OLTP assim que ele der entrada no pátio.

## 6.2 Especificação Funcional do Protótipo do Chatbot de Voz (IA Concierge)

Para atender à solicitação de especificação funcional do assistente virtual interativo por voz, detalha-se abaixo o fluxo de execução, as integrações de APIs e o tratamento de exceções.

### Fluxo Passo a Passo de Entrada e Saída

1. **Captura e Transmissão:** O motorista pressiona o botão de comando no volante e fala: *"Concierge, estou com fome, sugira uma pizzeria bem avaliada perto da minha rota atual."* O gateway de bordo captura o áudio (formato Opus/WAV) e transmite via MQTT para o servidor de IA.
2. **Transcrição (STT):** O servidor consome o áudio e aciona a **API OpenAI Whisper** para realizar o *Speech-to-Text*, convertendo o áudio na string de texto correspondente.
3. **Enriquecimento de Contexto (Busca Vetorial / RAG):** O microserviço de IA intercepta a requisição, lê as coordenadas geográficas atuais do carro no Redis (ex: Latitude -22.90, Longitude -43.17) e busca no banco de dados **MongoDB Atlas**



(utilizando *Vector Search* e similaridade de cosseno) os estabelecimentos classificados como "pizzaria" em um raio de até 5 km da posição atual do veículo, filtrando apenas aqueles com nota superior a 4.5 estrelas.

4. **Geração da Resposta Contextualizada (LLM):** O sistema monta um *prompt* estruturado contendo o histórico de conversas do usuário, as informações extraídas do MongoDB e as travas de comportamento do sistema, enviando-o para a **API do GPT-4o**.
5. **Síntese de Voz (TTS):** O texto de retorno da LLM é enviado para a **API OpenAI TTS (Text-to-Speech)**, gerando um arquivo de áudio sintetizado de voz ultra-realista.
6. **Execução e Navegação:** O áudio é reproduzido nos alto-falantes do veículo. Simultaneamente, um payload JSON contendo as coordenadas da pizzaria escolhida é injetado diretamente no sistema de navegação (GPS) do painel do carro para traçar a nova rota automaticamente.

### Exemplo Real de Interação (Cenário de Sucesso)

- **Usuário (Voz):** *"Concierge, indica um lugar bom para comer massa agora perto de onde estou."*
- **Prompt Interno Gerado (RAG):** Plaintext  
[Instrução]: Você é o Concierge de Viagem da Locadora. Responda de forma sucinta e amigável. Use estritamente o contexto fornecido abaixo. Não invente locais.  
[Contexto de Localização]: Veículo ID 4400 está na Av. Atlântica, Copacabana.  
[Dados do MongoDB]: 1. Don Camillo (Massa, Nota 4.8, Distância 600m). 2. La Trattoria (Massa, Nota 4.6, Distância 1.2km).  
[Pergunta do Usuário]: "indica um lugar bom para comer massa agora perto de onde estou."  
  
• **Concierge (Voz de Retorno):** *"Com certeza! Encontrei o restaurante Don Camillo, excelente para massas, avaliado com 4.8 estrelas e a apenas 600 metros de você na Avenida Atlântica. Já estou configurando a rota no seu GPS. Bom apetite!"*

### Tratamento de Erros e Exceções Funcionais

- **Cenário A: Queda de Conexão de Rede (Túnel / Sem Sinal Celular)**  
*Tratamento:* O Gateway de Borda monitora o *timeout* da rede. Se a requisição falhar por falta de sinal móvel, o sistema de bordo exibe uma mensagem visual no painel e emite um áudio local pré-gravado: *"Conexão indisponível no momento. Por favor, tente novamente assim que sairmos do túnel."* prevenindo que a aplicação trave.
- **Cenário B: Nenhum Local Encontrado no Raio de Busca (Falta de Dados)**  
*Tratamento:* Se a busca vetorial no MongoDB retornar vazia para o raio estipulado, a LLM é instruída por parâmetros rígidos a não inventar dados. O assistente responderá: *"Vasculhei a região atual, mas não encontrei opções de massa num raio de 5 quilômetros. Gostaria de tentar outra especialidade ou expandir a nossa busca?"*
- **Cenário C: Falha ou Lentidão nas APIs de IA Externas (Timeout)**  
*Tratamento:* O microserviço de IA possui um *Circuit Breaker* configurado para 3

segundos. Se a API externa de processamento falhar ou demorar a responder, o sistema aciona uma regra de contingência local que busca no Redis as rotas padrão e emite a resposta: *"Tive um problema ao processar o seu pedido de voz agora, mas mantive a sua rota principal ativa no painel."*

## 7 Governança, Segurança e Conformidade (LGPD)

Considerando o manuseio de dados sensíveis de localização e a integração do Assistente de IA com dispositivos pessoais, o projeto adota os princípios de *Privacy by Design* para garantir conformidade estrita com a Lei Geral de Proteção de Dados (LGPD).

- **Consentimento e Opt-In:** O funcionamento do Assistente de IA (Concierge) e a leitura de contexto (como a agenda do cliente) estarão bloqueados por padrão. O uso dependerá de um aceite explícito (*Opt-In*) no momento da assinatura do contrato de locação ou no aplicativo do usuário, podendo ser revogado a qualquer instante.
- **Anonimização de Dados:** Os dados brutos de GPS e rotas armazenados no *Data Lake* (Amazon S3) e no *Data Warehouse* (MySQL) para fins de treinamento de modelos analíticos serão anonimizados, dissociando completamente a rota histórica do CPF ou identidade do condutor.
- **Criptografia:** Todos os dados em repouso (*at rest*) nas camadas de armazenamento e os dados em trânsito (*in transit*) nas filas do Kafka e conexões WebSockets serão protegidos por protocolos avançados de criptografia (TLS/SSL para trânsito e AES-256 para disco), prevenindo o vazamento de informações corporativas e pessoais.

## 8 Estimativa de Custos Financeiros da Nuvem

Como a solução proposta está inteiramente baseada em computação em nuvem, mapeou-se os custos embutidos na operação do consórcio adotando majoritariamente o modelo de cobrança sob demanda (*Pay-as-you-go*) e instâncias reservadas para garantir previsibilidade financeira:

1. **Camada de Ingestão (Apache Kafka / Confluent Cloud):** Cobrança baseada no volume de dados trafegados (Gb/mês) e número de partições ativas. Estimativa de **US\$ 150.00 / mês** para processar a telemetria ininterrupta da frota inicial.
2. **Camada de Processamento (Databricks / Spark / Flink):** Cobrança por hora de cluster ativo baseada em unidades de computação (DBUs/hora). Utilizando políticas de *Auto-scaling* (desligando nós do cluster na madrugada caso não haja processamento batch rodando), estima-se o custo de **US\$ 850.00 / mês**.
3. **Camada de Armazenamento (Amazon S3 e NoSQL):**
  - *Amazon S3 (Data Lake):* Cobrança por GB armazenado (US\$ 0.023 por GB). Estimativa de **US\$ 80.00 / mês** (com políticas de ciclo de vida movendo dados antigos para camadas frias como o Glacier).

- *MongoDB Atlas e Redis Enterprise*: Camadas gerenciadas escaláveis baseadas em IOPS e armazenamento em RAM. Estimativa combinada de **US\$ 320.00 / mês**.

4. **APIs de Inteligência Artificial (OpenAI Whisper/GPT/TTS)**: Cobrança estritamente baseada em volume de uso (Preço por milhão de *tokens* de texto e preço por minuto de áudio processado). Estimando uma média de 5 interações por dia por veículo locado, projeta-se um custo variável de **US\$ 0.05 por locação ativa**.

O custo fixo mensal de infraestrutura em nuvem estabiliza-se em aproximadamente **US\$ 1,400.00 / mês**, valor facilmente absorvido pelo consórcio dado o ganho de eficiência operacional com a manutenção preditiva e a monetização do pacote de serviços *premium* do Concierge de Voz.

## 9 Conclusão

A arquitetura proposta nesta Parte III representa um salto tecnológico fundamental e disruptivo para o consórcio de locadoras. Ao transcender o modelo tradicional de *Data Warehouse* — desenhado e implementado nas Partes I e II para responder a perguntas predominantemente históricas, estáticas e estruturadas —, a integração holística de Internet das Coisas (IoT) e Big Data consolida um ecossistema proativo, capaz de "escutar", interpretar e reagir à frota de veículos conectados em tempo real.

Os objetivos iniciais que motivaram esta evolução foram plenamente atendidos e blindados pelas escolhas tecnológicas fundamentadas ao longo de toda a proposta:

- A adoção da **Arquitetura Lambda**, amparada pelo Apache Kafka para ingestão resiliente e motores duros de processamento paralelo (Apache Spark para manutenção preditiva e Apache Flink para eventos instantâneos), garantiu o **Monitoramento Operacional Instantâneo**, permitindo reações rápidas a acidentes através do Painel de Emergências do Dashboard.
- A persistência poliglota escalável (utilizando Cassandra para alta taxa de escrita e Redis para baixíssima latência em memória) viabilizou a leitura contínua de padrões de direção, fornecendo o suporte exato para o **Painel de Cobranças e Faturamento Dinâmico/Gamificação**.
- A introdução de Inteligência Artificial Generativa controlada estritamente pela técnica de **RAG (Retrieval-Augmented Generation)** e Busca Vetorial no MongoDB Atlas tornou viável o **Assistente de IA (Concierge de Viagem)**, fornecendo uma especificação funcional segura, integrada com APIs de mercado, livre de alucinações e em total conformidade com a LGPD.

Por fim, é essencial destacar que este novo ecossistema de Big Data não descarta e nem invalida o legado analítico construído nas etapas anteriores; pelo contrário, ele o enriquece. O fluxo de dados históricos (*Cold Path*) consolidado no *Delta Lake* retroalimentando periodicamente o banco analítico em MySQL prova que é perfeitamente viável aliar a governança e estabilidade dos dados relacionais tradicionais com a agilidade, flexibilidade e escala massiva do *streaming*. Com esta infraestrutura unificada, robusta e financeiramente otimizada, o consórcio mitiga seus riscos operacionais, reduz drasticamente custos de manutenção na oficina e assegura a sua posição na vanguarda da mobilidade inteligente.